FOLOSIYE ADEGBOLA

PROJECT REPORT

C00286234

Table of Contents

1. Introduction 4
2. Areas and technologies Researched. 5
2.1 Overview of Intrusion Detection Systems 5
2.2 Signature-Based NIDS 6
2.2.1 Operating Principle 6
2.2.2 Strengths of Signature Based NIDS 7
2.2.3 Limitations & Weaknesses 7
2.2.4 Why Signature-Based NIDS Alone Are Not Enough 8
2.3 ML-based (Anomaly / Behavioural) NIDS — Theory, Strengths & Weaknesses 8
2.3.1 What is ML-based / Anomaly-based IDS/NIDS 8
2.3.2 Evidence: What Research Says (Strengths) 9
2.3.3 Challenges & Weaknesses of ML-based NIDS 9
2.3.4 Why ML-based NIDS (with XAI) 10
2.3.5 Summary 10
2.4 Datasets, Preprocessing & Feature Engineering 11
2.4.1 Dataset Description — CIC-IDS2017 11
2.4.2 Known Problems & Data Quality Criticisms 11
2.4.3 Data-Processing & Flow Re-Extraction Plan 12
2.4.4 Justification & Rationale 13
2.4.5 Risks & Challenges 14
2.5 Machine Learning Models and Explainability Integration 14
2.5.1 Choice of ML Models: Rationale 14

2.5.2 Integration of Explainable AI (XAI): Why and How 15
2.5.4 Implementation Considerations & Pipeline Flow 15
2.5.5 Justification for This Approach 16
2.6 Summary 16
3. System Implementation 17
3.1 System Architecture Overview 17
3.1.1 Data Ingestion Layer 17
3.1.2 Detection Layer (ML Inference Engine) 17
3.1.3 Explainability Engine (XAI Integration) 17
3.1.4 Monitoring and Dashboard Layer 18
3.1.5 Logging and Audit Layer 18
3.2 Model Deployment and Serving 18
3.3 Pipeline Workflow 19
3.4 Technical Stack 19
3.5 Justification of Design 19
4. Testing and Evaluation Strategy 20
4.1 Evaluation Objectives 20
4.2 Planned Test Setup 20
4.2.1 Hardware and Runtime Environment 20
4.2.2 Dataset Partitioning 21
4.3 Performance Metrics 21
4.4 Explainability Validation 21
4.5 Operational Testing 21
4.6 Expected Outcomes 22
4.7 Summary 22
5. Security Considerations 22
5.1 Threat Model 23
5.2 Model Robustness 23
5.2.1 Adversarial Evasion 23
5.2.2 Data Poisoning 23
5.3 Explainability Risks 24
5.4 Operational Security 24

5.5 Privacy Considerations 24
5.6 Summary 25
6. Limitations 25
6.1 Dataset Limitations 25
6.1.1 Simulated Environment 25
6.1.2 Label Accuracy and Flow Integrity 25
6.2 Model Limitations 26
6.2.1 Overfitting to Dataset-Specific Patterns 26
6.2.2 Sensitivity to Feature Noise 26
6.3 Explainability Constraints 26
6.3.1 XAI Fidelity and Trust 26
6.3.2 Computation Overhead 26
6.4 Operational and Architectural Limitations 26
6.5 Evaluation Limitations 27
6.6 Summary 27
7. Conclusion and Future Work 27
7.1 Project Summary 27
7.2 Project State and Next Steps 28
7.3 Future Development Opportunities 28
7.4 Final Remarks 28

1. Introduction

Modern organizations operate in an environment where cyber threats and attacks are increasing in number, sophistication, and unpredictability. Networks today face a continuous stream of malicious activity ranging from simple reconnaissance scans to coordinated, multi-stage intrusions involving malware, encrypted communication channels, and stealthy data exfiltration techniques which have begun to involve state to state attacks. In this landscape, quick and reliable detection mechanisms are essential for maintaining the security posture of any digital infrastructure.

Network Intrusion Detection Systems (NIDS) play a key role in this defence strategy by monitoring network traffic to identify suspicious activity. Traditional NIDS rely mainly on

signature-based detection, where patterns of known attacks are matched against network flows. While effective for identifying previously catalogued threats, these systems face inherent limitations when confronted with zero-day attacks, polymorphic malware, evasion techniques, and behaviours that do not fit predefined rules. This reactive nature leaves a gap between emerging threats and the availability of corresponding signatures.

Machine learning has emerged as a promising alternative due to its ability to generalize from historical data and detect behavioural anomalies rather than exact signatures. ML-based NIDS can identify both known and previously unseen attacks, offering adaptability traditional approaches may lack. However, these systems introduce their own challenges, most notably a lack of interpretability. Security analysts often struggle to understand why a model has flagged a particular network flow as malicious, which can undermine trust and hinder incident response.

Explainable Artificial Intelligence (XAI) addresses this issue by providing insights into how ML models reach their decisions. Techniques such as SHAP and LIME reveal feature contributions and help analysts interpret alerts in a meaningful way. Incorporating XAI into ML-based NIDS offers a balanced solution: stronger detection capabilities combined with transparency and analyst friendly explanations.

This research report examines the theoretical, technical, and practical foundations required to build a machine-learning-based NIDS enhanced with interpretability mechanisms. It evaluates relevant datasets, algorithms, preprocessing techniques, model evaluation strategies, and operational technologies such as FastAPI, Prometheus, and Grafana. The aim is to establish a strong research basis that informs the design and implementation of the final system. Each component of the proposed solution is aligned through literature, comparative analysis, and alignment with modern cybersecurity needs.

2. Areas and technologies Researched.

2.1 Overview of Intrusion Detection Systems

Intrusion Detection Systems (IDS) are security mechanisms designed to monitor, analyse, and identify malicious or policy-violating activity within an information system. They serve as an essential layer of defence, complementing firewalls, endpoint controls, and incident response processes. IDS technologies are typically categorized into two main forms: Host-Based IDS (HIDS) and Network-Based IDS (NIDS). HIDS focuses on activity occurring within individual devices, such as file modifications, process events, and privilege escalations. In contrast, NIDS observes traffic flowing across network segments, enabling it to detect patterns of scanning, malware communication, lateral movement, and denial-of-service activity.
NIDS solutions operate by analysing packet payloads, flow-level metadata, or aggregated behavioural patterns. Traditional NIDS relies heavily on signature-based

detection, where known malicious patterns are compared against live traffic. This method has been popular for decades due to its precision and low false-positive rates when signatures are accurate. Well-established tools such as Snort and Suricata exemplify this approach, using curated rule sets to detect millions of known attack variations.

Despite their success, signature-based systems inherently struggle with modern threat landscapes. Attackers constantly develop new techniques, mutate payloads, and use encryption or obfuscation to bypass static signatures. Zero-day attacks remain undetectable until a signature is created and distributed. This delay between discovery and protection introduces “blind periods” where networks are exposed. Additionally, maintaining large rule repositories becomes resource-intensive and prone to misconfiguration.

These challenges have accelerated interest in anomaly-based and ML-enhanced IDS models, which instead learn normal behaviour and identify deviations. Unlike static rules, machine learning models analyse network features, statistical patterns, and historical traffic trends to detect unusual or malicious activity. This paradigm aims to expand detection capability beyond known attacks, offering more adaptive and proactive protection.

The growing complexity of cyber threats, combined with the limitations of traditional rule-based mechanisms, has positioned ML-based NIDS as a promising direction for future security architectures. However, these approaches introduce their own barriers — notably interpretability, model trust, and dataset quality — which must be evaluated carefully before deployment. The subsequent sections explore these elements in depth, laying the research foundation for a machine-learning-based NIDS with explainable decision-making.

2.2 Signature-Based NIDS

Signature based Network Intrusion Detection Systems are the most widely used form of Network Intrusion Detection Systems, mainly in legacy or resource limited environments due to their simplicity and reliability against known threats. This section serves to highlight why despite the flaws associated with Signature based Network Intrusion Detection Systems, they are still reference points and why a Machine Learning Based approach can overcome their flaw

2.2.1 Operating Principle

- Signature based Network Intrusion Detection Systems operate on a similar level antivirus software. It analyses network traffic against a database of

known signatures or patterns associated with known threats. It relies on signatures which can be a specific byte pattern, payload signature, sequence of packets or any known indicator of compromise.

- As traffic flows through the monitoring point, each packet or flow is compared against the signature database. If a match occurs the IDS flags the traffic as potentially malicious or malicious, an alert is raised and depending on configurations and deployment data is logged.
- The detection is rule based. The system relies solely on exact patterns matching known threats, yielding predictable and reproducible results.
- Some examples would be snort, which uses an open-source signature-based system. It uses its rule language to define malicious patterns and can operate in IDS mode or IPS mode. Another would be suricata which supports signature-based detection and optionally combines with anomaly-based methods.

2.2.2 Strengths of Signature Based NIDS

Signature based approaches are still popular for good reasons:

- There are high accuracy and low false positives for known attacks. Due to the detection being based on precise patterns, false alarms are rare when the signatures are correct and comprehensive
- Pattern matching is computationally cheap compared to statistical or ML based methods, allowing near real time traffic inspection.
- The behaviour is predictive. What is found in the signature database gets detected.

2.2.3 Limitations & Weaknesses

- Inherently reactive to known threats. Its core detection logic is based on matching prior knowledge; therefore, any novel attack variant or sophisticated evasion that alters its known pattern will bypass detection.
- The signature databases must be constantly updated. New threats would require manual analysis, signature generation, and deployment.
- As the signature set grows, the quality of performance may be affected or require more resources.
- Between the time a new attack emerges and when its signature is captured, analysed and deployed the network would remain vulnerable

2.2.4 Why Signature-Based NIDS Alone Are Not Enough

In the context of today's networks — high volume, encrypted traffic, polymorphic malware, zero-day exploits — signature-based NIDS show serious gaps. While they are excellent for known threats, they offer poor coverage when attackers innovate or reuse old techniques with minor variations or operate under encryption/obfuscation.

Studies comparing signature-based and anomaly-based detection show that while signature-based systems have low false positives and high accuracy for known threats, they fall short in detecting novel or evolving threats; anomaly-based or ML-based approaches offer better adaptability at the cost of more false positives and higher resource demand.

Given these limitations, many researchers (and industry practitioners) advocate for hybrid approaches that combine signature-based detection for known threats and anomaly/ML-based detection for unknown threats — providing layered defence that covers both known and novel threats.

2.3 ML-based (Anomaly / Behavioural) NIDS — Theory, Strengths & Weaknesses

2.3.1 What is ML-based / Anomaly-based IDS/NIDS

- Unlike signature-based NIDS, which match traffic against known attack patterns, anomaly-based (or ML-driven) NIDS aim to learn normal network behaviour (via statistical, machine learning or data-driven models) and detect deviations as possible intrusions.
- Such systems model "normal" flows (or a combination of benign and attack flows in supervised setups), and flag flows or traffic patterns that deviate significantly from learned norms.
- Over time, with appropriate data and retraining, ML-based NIDS can adapt to evolving network behaviours and emerging attack patterns — including zero-day or previously unseen attacks.

  Because of those properties, ML-based NIDS are often referred to in literature as better suited for dynamic, modern networks — where traffic types, user behaviour, and threat vectors evolve quickly.

2.3.2 Evidence: What Research Says (Strengths)

Multiple recent studies confirm the advantages of ML-based NIDS:

- A 2025 study showed that tree-based models (e.g. Random Forest, Decision Trees) delivered extremely high accuracy (near 0.9996 / 0.9997) and AUC = 1.00 on benchmark datasets, with precision > 0.99 — while remaining computationally efficient compared with deep-learning alternatives.
- ML-based NIDS have been documented to provide improved detection accuracy, reduced false positives, and the ability to detect previously unseen attack vectors.
- A broad survey of ML-based IDS emphasizes that ML enables detection of a wide variety of attacks (DoS, reconnaissance, brute force, malware traffic, insider threats) across traffic types, often outperforming traditional IDS in adaptability and coverage.

These findings support the rationale for using ML — especially classical ML (tree-based, ensemble methods) on flow-level datasets — as a realistic and effective NIDS in real-world environments.

Furthermore, compared to signature-based systems, ML-based NIDS reduce the dependence on continuously updated signature databases and manual rule authorship, which significantly reduces maintenance burden and enhances responsiveness to novel threats.

2.3.3 Challenges & Weaknesses of ML-based NIDS

Despite advantages, ML-based NIDS are not a silver bullet. Key challenges:

- False Positives / False Negatives Trade-off: Anomaly-based detection flags deviations — but not all deviations are malicious. High variability in “normal” behaviour can lead to many false positives.
- Dataset Quality & Representativeness: ML models are as good as the data they are trained on. If datasets are outdated, biased, lack realistic noise or don’t reflect actual network behaviours, model performance in real-world deployment degrades.
- Computational & Resource Overhead: Depending on algorithm and data volume, ML-based systems can be resource-intensive — especially for real-time detection or high-throughput networks.
- Interpretability / Explainability Issues: ML models (especially complex ensembles or deep learning) often act as black boxes, making it difficult for analysts to understand — and trust — why a flow was flagged.

- Maintenance & Concept Drift: Network behaviour evolves new protocols, legitimate pattern changes which can degrade model accuracy over time unless retrained. Also, adversarial actors may adapt to evade ML models.

Because of these issues, deploying a pure ML-based NIDS in production requires careful design: good data, continuous evaluation, possibly hybrid methods, and mechanisms for explanation & analyst oversight.

2.3.4 Why ML-based NIDS (with XAI)

Given the trade-offs above, there are strong justifications for choosing an ML-based NIDS for your project — especially given your design goals:

- Coverage beyond known attacks: Your system targets not only catalogued threats but also zero-day or novel attacks — where signatures don't yet exist. ML-based detection enables that.
- Adaptability to evolving traffic & attacks: With periodic retraining, your system can adjust to changing network patterns and attacker behaviours.
- Competitive performance with classical ML: Given recent research — e.g. tree-based models achieving high accuracy and AUC — your plan (Random Forest / XGBoost) is realistic and effective.
- Feasibility on flow-level data: Flow-based datasets (e.g. from CIC-IDS) suit classical ML well — less processing overhead than full packet inspection, easier feature extraction, more privacy-respecting.
- Opportunity for Explainability (XAI): Combining ML with explainability techniques (e.g. SHAP/LIME) directly addresses the biggest weakness: "Why was this flow flagged?" → builds trust and analyst interpretability.

Thus, ML-based NIDS with XAI offers a balanced compromise: detection strength + explainability + manageable resource demand

2.3.5 Summary

Use ML-based NIDS when:

- You expect novel, evolving threats (zero-day, polymorphic, encrypted)
- You have access to good flow-level datasets and compute resources
- You need adaptability and lower maintenance than signature-rule management
- You value explainability and analyst transparency

Avoid (or supplement with signature-based) when:

- Attack patterns are well-known and signature databases are reliable
- Network traffic is extremely high-throughput and real-time constraints are tight
- Dataset quality is poor or unrepresentative
- You can’t guarantee regular retraining or monitoring

2.4 Datasets, Preprocessing & Feature Engineering

2.4.1 Dataset Description — CIC-IDS2017

- CIC-IDS2017 is a publicly available dataset designed for intrusion detection research — it includes both raw packet-capture files (PCAP) and pre-extracted flow-based features (CSV) generated by a flow exporter.
- The dataset contains approximately 2.83 million flow instances, collected over a 5-day period (3–7 July 2017).
- It covers benign (normal) traffic plus a variety of attack types, such as Brute Force (FTP/SSH), DoS / DDoS, Web Attacks, Botnet, Infiltration, etc.
- The flow-based CSV contains up to 84 features per flow, covering bidirectional flow statistics (e.g. packet counts, bytes, durations, flag counts, inter-arrival times, and more) — making it well-suited for machine learning classification.

Because of its size, variety of attack types, availability of both raw and processed data, and wide adoption in academic ML/NIDS research — using CIC-IDS2017 makes work comparable to many prior studies.

2.4.2 Known Problems & Data Quality Criticisms

Recent analyses have uncovered significant limitations in the publicly distributed flow-based data of CIC-IDS2017:

- A detailed study showed that flows generated by the original flow-exporter (CICFlowMeter) contain multiple errors: feature-duplication, incorrect feature calculations, erroneous protocol detection, inconsistent TCP flow terminations, and questionable labeling.
- As a remedy, the authors of that study proposed a re-extraction tool (LycoSTand) to rebuild flows from raw PCAPs and correct dataset flaws. Benchmarks using the corrected dataset showed significant performance improvements for ML models compared to the original CSV-based dataset.

- Other criticism argues that many published ML-based NIDS relying solely on the summarized CSV flows are unlikely to hold up in practical deployments; summarization removes packet-level detail that might matter, and dataset artefacts inflate performance metrics.
- More broadly, an analysis published recently (2024) shows that dataset creation, flow exporter choices, and flow-generation methodology significantly impact ML model performance and generalizability — meaning that classifier results on flawed data may be misleading.

2.4.3 Data-Processing & Flow Re-Extraction Plan

To mitigate dataset flaws and maximize validity, this pipeline will be followed

1. Obtain raw PCAP data from CIC-IDS2017
2. Reconstruct flows using a validated flow-exporter
    - Preferably use a corrected or updated flow-exporter (e.g. LycoSTand) rather than the original CICFlowMeter output. This ensures corrected flow construction, accurate feature calculation, and proper labeling of bidirectional flows.
    - Ensure exporter version is documented for reproducibility.
3. Inspect & validate flows (data cleaning / sanity-checks):
    - Discard flows with zero packets, zero duration, missing fields, or obviously invalid metadata.
    - Remove duplicates or redundant flows.
    - Check protocol distributions, port / flag consistency (especially for TCP flows), and ensure the flow-termination logic (SYN-FIN or timeout) is valid.
4. Feature normalization / standardization:
    - For continuous numerical features (packet counts, bytes, durations, inter-arrival times, etc.), apply scaling (e.g. Min-Max or Standard scaling).
    - For categorical features (protocol, flags, ports if used), apply appropriate encoding (label or one-hot).
5. Class imbalance handling (attack vs benign; per-attack class imbalance):
    - Use oversampling techniques (e.g. SMOTE) to synthetically balance minority classes (especially rare attack types).

- Optionally, combine oversampling with undersampling of benign flows to keep dataset size manageable.
- Ensure stratified train/validation/test split, preserving attack/benign proportions (or balanced where needed).

6. Feature selection / dimensionality reduction:
    - Use tree-based feature-importance (from preliminary models like Random Forest or XGBoost) to identify and drop irrelevant / redundant features.
    - Optionally, apply dimensionality-reduction techniques (e.g. PCA, Autoencoder) — but only after verifying that reduction doesn't remove meaningful signal for detection or interpretability (because you need explainability).
    - Keep detailed logs of which features are used / removed (for documentation).
7. Dataset split & validation set preparation:
    - Use stratified splitting (e.g. 70/15/15 for train/validation/test) or k-fold cross-validation, ensuring each split contains representative attack and benign samples.
    - Maintain metadata mapping (flow-ID, timestamps, original PCAP references) internally — for potential future traceability or forensic analysis.
8. Document data provenance & transformations:
    - For reproducibility, record all processing steps in notebooks / scripts (with versioning), including exporter version, cleaning criteria, encoding/scaling parameters, sampling strategy, feature selection, and splits.

2.4.4 Justification & Rationale

- Re-extracting flows from raw PCAP ensures you avoid known export and labeling errors — boosting the credibility and reliability of your results.
- Cleaning and balancing the data helps prevent skewed or over-optimistic ML performance (common when benign flows dominate) and ensures minority attack classes are properly learned.
- Feature selection / reduction reduces noise, improves training speed, and lessens risk of overfitting especially important given the large number of original features.
- Documenting the entire pipeline supports reproducibility, transparency, and academic rigour traits expected in a top-grade research report.
- acknowledging dataset limitations explicitly (even after cleaning)

2.4.5 Risks & Challenges

Even with the above pipeline, some risks remain:

- Some flows even after re-extraction may still not reflect real-world production network traffic (noise, background traffic, encrypted traffic, protocol variations). CIC-IDS2017 was generated in a controlled environment.
- Oversampling (e.g. SMOTE) can produce synthetic flows that never actually occurred which may distort real-world applicability.
- Feature-selection or dimensionality-reduction may inadvertently remove subtle signals important for rare or complex attack detection.
- Performance on this “cleaned, balanced, curated” dataset may overestimate real-world detection rates meaning conclusions must remain conservative and caveated.

2.5 Machine Learning Models and Explainability Integration

2.5.1 Choice of ML Models: Rationale

A range of machine learning algorithms is suitable for classification tasks on network-flow data; tree-based models and ensemble learners have demonstrated consistent performance in intrusion detection contexts.

For this project, the primary models selected are:

- Random Forest (RF) — chosen for robustness to noise, ability to handle mixed data types (numerical and categorical), resistance to overfitting for high-dimensional datasets, and relative interpretability compared to more opaque models.
- XGBoost (Extreme Gradient Boosting) — selected for high classification performance, efficiency with large datasets, and suitability for flow-based NIDS data. Gradient-boosted decision trees have become a standard in recent intrusion detection literature.

These models strike a balance between detection performance, computational efficiency, and compatibility with explainability techniques — a critical requirement of this project.

Additionally, simpler or alternative algorithms (e.g. k-Nearest Neighbours, Support Vector Machines, decision trees) are considered for baseline comparison or if conditions (e.g. feature set, imbalance) warrant. Surveys of modern IDS research show that lightweight models retain value, especially when computational or resource constraints exist.

The aim is to develop a pipeline where models produce high-quality predictions while remaining compatible with explainability — enabling transparent decision-making by analysts and supporting trust in automated detection.

2.5.2 Integration of Explainable AI (XAI): Why and How

Traditional ML models, especially when used in security-critical systems such as NIDS, often suffer from the "black-box" problem: their predictions may lack intuitive, human-understandable justifications. This undermines analyst trust, complicates incident investigation, and reduces operational value.

To address this, the project integrates explainable AI methods. Two widely adopted techniques in intrusion detection contexts are:

- SHAP (SHapley Additive exPlanations) — computes feature-contribution values (Shapley values) for each prediction, allowing both *global* (feature-importance across dataset) and *local* (why this flow was flagged) interpretability. SHAP has become a standard when interpretability is required.
- LIME (Local Interpretable Model-agnostic Explanations) — produces localized surrogate models around a specific prediction to approximate which features most influenced it, offering an alternative, model-agnostic explanation. LIME remains useful for instance-level explanations, especially in combination with other XAI methods.

Recent studies demonstrate that combining SHAP and LIME leads to more robust and trustworthy explainability in NIDS: SHAP provides consistent global feature-importance attribution while LIME enables flexible local explanations, yielding a complementary explanatory setup.

Consequently, once the RF / XGBoost models are trained on the cleaned, pre-processed flow-based dataset, their predictions will be annotated with SHAP and/or LIME outputs. These explanations will be exposed to the analyst via the system's monitoring dashboard (see overall architecture), enabling human-in-the-loop review, justification of alerts, and improved transparency.

2.5.4 Implementation Considerations & Pipeline Flow

The integration pipeline will follow these stages:

- Pre-processed dataset train/test split.
- Train ML models: Random Forest and XGBoost. Perform hyperparameter tuning (grid search / random search) to optimise performance (without overfitting).

- Evaluate trained models on held-out test data using classification metrics above.
- For each prediction (or for a sample subset): generate SHAP explanations (global + local) and LIME explanations (local). Store explanation metadata (feature-contribution values, feature rankings) alongside prediction output.
- Integrate predictions + explanations into the API / dashboard layer (FastAPI + monitoring) so that each alert includes justification: feature-based reasonings that the analyst can inspect.
- Log explanations and model decisions for auditing, forensic inspection, and potential feedback/training loops (e.g. analyst overrides, retraining with corrected labels).

2.5.5 Justification for This Approach

While deep learning models (e.g. neural networks) are increasingly popular in intrusion detection research, they often suffer from lower interpretability and higher computational cost — making explainability integration more complex or inefficient.

By using tree-based ensemble models (RF, XGBoost) — which already perform well on tabular, flow-based data — and combining them with model-agnostic XAI methods (SHAP, LIME), this design achieves high detection performance, low resource overhead, and transparent decision-making. This balance supports not only academic rigor but also practical applicability in realistic environments where analyst trust and auditability matter.

Additionally, this approach aligns with frameworks proposed in recent literature, which argue for hybrid models that combine high performance with human-interpretable output.

2.6 Summary

This chapter reviewed existing intrusion detection system paradigms, covering both signature-based and machine learning-based approaches. It outlined their respective strengths and limitations in the context of evolving cyber threats, particularly in high-volume, encrypted, and polymorphic traffic environments. The justification for adopting a flow-based ML NIDS combined with XAI techniques was presented in detail, supported by recent literature and empirical results from the domain.

CIC-IDS2017 was selected as the primary dataset for the implementation phase, with a careful reprocessing and cleaning strategy to mitigate known flaws. The Random Forest and XGBoost models were chosen based on their balance between accuracy, speed, and

explainability. Explainability will be delivered through SHAP and LIME to ensure alerts are interpretable by human analysts.

The next chapter will describe the implementation of the system, including the detection pipeline, XAI integration, and operational components such as the monitoring stack and logging mechanisms.

3. System Implementation

This chapter outlines the architecture, design, and implementation of the proposed machine learning-based Network Intrusion Detection System (NIDS). It details the system's modular structure, the flow of data from ingestion to detection and explanation, and the integration of monitoring and analyst-facing components.

3.1 System Architecture Overview

The proposed system is composed of five primary layers:

3.1.1 Data Ingestion Layer

- Handles the loading of pre-processed flow-based data derived from CIC-IDS2017.
- Acts as the entry point for flow records in CSV format for real-time or batch classification.

3.1.2 Detection Layer (ML Inference Engine)

- Contains trained ML models (Random Forest, XGBoost).
- Accepts incoming flows, performs feature extraction if needed, and outputs binary (benign/attack) or multi-class predictions.
- Prediction results are tagged with flow metadata for downstream processing.

3.1.3 Explainability Engine (XAI Integration)

- Applies SHAP and LIME to output local and global explanations of predictions.
- For each alert, generates a ranked list of features contributing to the model’s decision.

3.1.4 Monitoring and Dashboard Layer

- Uses FastAPI to serve predictions and explanations to frontend.
- Integrates with Prometheus (metrics scraping) and Grafana (visualization).
- Displays incoming flow statistics, alert summaries, explanation heatmaps, and model behaviour.

Grafana Dashboard Example:



3.1.5 Logging and Audit Layer

- Stores all flow inputs, prediction outcomes, SHAP/LIME outputs, timestamps, and confidence scores.
- Supports forensic traceability and future feedback-loop training.

3.2 Model Deployment and Serving

Models are pre-trained offline using the processed dataset, then deployed in serialized format (e.g., via joblib or pickle) and loaded into a lightweight inference API server:

- FastAPI is used to expose REST endpoints for submitting flows and retrieving predictions with explanations.
- Model inference latency is minimized through batch input support and concurrent worker threads.
- SHAP values are cached per class to improve response speed for repetitive flows.

3.3 Pipeline Workflow

The system operates in the following logical sequence:

- Ingest new flow record(s) via API or file upload.
- Standardize and align features (as per training schema).
- Submit features to trained ML model (RF/XGBoost).
- Capture prediction output + probability/confidence score.
- Trigger SHAP/LIME explainer engine to generate explanation vector.
- Push result (flow, prediction, explanation) to monitoring/logging services.
- Display live on Grafana dashboard or store for analyst review.

3.4 Technical Stack

Component	Technology Used
ML Model Training	Python, Scikit-learn, XGBoost
Data Handling	Pandas, NumPy
API Layer	FastAPI
Explainability	SHAP, LIME
Monitoring	Prometheus + Grafana
Logging	CSV exports JSON logs
Development Environment	Jupyter, VSCode

3.5 Justification of Design

- Flow-based approach was selected for its efficiency, privacy preservation, and realistic alignment with network telemetry.
- FastAPI was chosen for its lightweight nature, asynchronous performance, and ease of integration.
- Prometheus + Grafana provides a widely used, DevOps-compatible solution for metrics and visualization, allowing real-time alert tracking.
- SHAP and LIME were selected due to their compatibility with tabular models and support in recent XAI/NIDS literature.
- Modular structure enables future extensions (e.g., live packet capture, unsupervised anomaly detection, retraining, feedback loops)

4. Testing and Evaluation Strategy

This chapter defines the evaluation criteria and testing procedures intended for assessing the performance, transparency, and operational behaviour of the machine learning-based Network Intrusion Detection System (NIDS). The focus is on validating the system's capability to accurately classify network flows, generate meaningful explanations for security alerts, and operate within the constraints of a near-real-time environment.

4.1 Evaluation Objectives

The testing phase of this project aims to answer the following key questions:

1. Detection Performance
    - How accurately does the system distinguish between benign and malicious network flows?
    - Does it generalise across different attack types in CIC-IDS2017?
2. Explainability Quality
    - Are the SHAP and LIME outputs interpretable and relevant to security analysts?
    - Do explanations align with actual attack indicators?
3. Operational Efficiency
    - Is the system responsive enough for near-real-time classification?
    - Can it support sustained inference and explanation workloads without bottlenecks?

4.2 Planned Test Setup

4.2.1 Hardware and Runtime Environment

Testing will be conducted on a local development environment configured with:

- CPU: Quad-core or higher
- RAM: Minimum 8 GB
- OS: Linux or Windows
- Python 3.11 with virtual environment
- Required libraries: scikit-learn, xgboost, pandas, SHAP, LIME, FastAPI, Prometheus

Optionally, Docker containers may be used for isolated deployment and API testing.

4.2.2 Dataset Partitioning

The reprocessed CIC-IDS2017 dataset will be used for all tests. It will be divided into:

- Training Set: 70% of labelled, cleaned flows
- Validation Set: 15% for hyperparameter tuning
- Test Set: 15% for final performance evaluation

All partitions will be stratified to maintain proportional representation of benign and attack classes.

4.3 Performance Metrics

To measure the effectiveness of the ML classifiers, the following metrics will be computed:

- Accuracy: Proportion of correctly classified flows
- Precision & Recall: To assess false positives and false negatives
- F1-Score: For balanced evaluation
- Confusion Matrix: For per-class inspection
- ROC-AUC: To measure class discrimination ability

These metrics will be computed separately for Random Forest and XGBoost models.

4.4 Explainability Validation

The quality of SHAP and LIME outputs will be evaluated through:

- Visual inspection of global feature importance (SHAP summary plots)
- Review of local explanations for individual flows flagged as attacks
- Comparison between explanations and known attack signatures from CIC-IDS2017

A small set of flows will be manually analysed to determine whether the XAI outputs highlight plausible security features.

4.5 Operational Testing

To verify that the system performs efficiently:

- Inference Latency Tests: Measure the time required to classify a batch of 1000 flows.
- Explanation Overhead: Evaluate the average time taken by SHAP and LIME to produce per-flow explanations.
- API Stress Tests: Simulate concurrent requests to the FastAPI endpoints and observe system behaviour under load.
- Dashboard Monitoring: Validate that Prometheus and Grafana correctly visualise incoming alerts, metrics, and system health.

4.6 Expected Outcomes

- Random Forest and XGBoost are expected to achieve high precision and recall, especially for dominant attack classes.
- SHAP should provide interpretable global feature rankings that align with known attack vectors (e.g., port usage, packet counts, flag ratios).
- LIME is expected to enhance analyst interpretability on a per-alert basis.
- Overall latency for prediction + explanation should remain within a usable range (<1s per flow in batch mode).
- Dashboard should provide usable real-time visibility into system behaviour.

4.7 Summary

The testing phase will focus on validating the detection performance, interpretability, and operational responsiveness of the ML-based NIDS. Results will be reported once the testing phase is complete and will inform the final evaluation of system reliability and practical deployment viability.

5. Security Considerations

This chapter discusses the security implications of the system's design, deployment context, and operational use. Given that this project focuses on intrusion detection — a core component of cybersecurity infrastructure — particular attention is paid to the risks posed by adversarial inputs, model misuse, and deployment environment constraints.

5.1 Threat Model

The system is designed to operate in a passive monitoring capacity, analysing flow-based network telemetry for potential signs of intrusion. It assumes:

- Access to NetFlow-like data (e.g., IP addresses, ports, packet counts, durations) but not raw packet payloads.
- Attackers may attempt to bypass detection by modifying flow characteristics.
- The ML models and XAI outputs are trusted, and not subject to manipulation unless the host environment is compromised.

Threats considered:

- Evasion through traffic obfuscation
- Adversarial perturbation of flow features
- Poisoning or biasing of training data
- Denial-of-Service against the monitoring or API pipeline
- Inference leakage through model responses

5.2 Model Robustness

5.2.1 Adversarial Evasion

Machine learning classifiers — particularly those trained on static datasets — are vulnerable to evasion attacks, where adversaries craft traffic that mimics benign patterns while delivering malicious payloads. This system mitigates this by:

- Using reprocessed, cleaned flow data for higher feature fidelity
- Incorporating explainability tools to inspect why a prediction was made, potentially uncovering “unusual benign” traffic
- Allowing analyst override or feedback in response to suspicious but undetected flows

Further defences (e.g., adversarial training or uncertainty-aware models) could be integrated in future work.

5.2.2 Data Poisoning

If the training data pipeline were compromised (e.g., labels manipulated), it could result in learned behaviours that misclassify malicious activity. This is mitigated by:

- Using publicly vetted datasets (CIC-IDS2017)

- Offline model training in controlled environments
- Avoiding live feedback-based retraining in current scope (reducing risk of poisoning)

5.3 Explainability Risks

Explainable AI increases transparency — but also introduces risks:

- Attackers may use SHAP/LIME outputs to understand decision boundaries and fine-tune evasion attacks
- Over-reliance on XAI may lead to false analyst trust in predictions

To address this:

- Explanations are intended for human interpretation, not automation
- Alerts remain subject to analyst judgement
- Logging includes both raw features and explanation vectors for auditing

5.4 Operational Security

The system operates in a read-only flow monitoring mode, meaning it does not alter network traffic or introduce inline decision-making. However, deployment security must consider:

- API endpoints should be protected via authentication and rate-limiting
- SHAP and LIME engines should not leak input flows in explanations returned to untrusted clients
- Logging systems should avoid storing sensitive data such as IPs or timestamps without access controls

5.5 Privacy Considerations

By using flow-level metadata instead of packet payloads, this system inherently reduces privacy exposure. Nonetheless:

- IP addresses, port numbers, and traffic patterns may still be considered personally identifiable information (PII) in regulated environments
- Logging and monitoring outputs should be anonymised or stored in encrypted formats

- The system does not perform any DPI (Deep Packet Inspection), avoiding direct content analysis

5.6 Summary

The proposed NIDS maintains a strong security posture by limiting its attack surface, avoiding inline interference, and focusing on explainable offline detection. Future iterations may explore adversarial training, differential privacy in model inference, or integration into hardened production pipelines. Nonetheless, even in its current form, the system introduces minimal additional risk while enabling effective and interpretable intrusion detection.

6. Limitations

While the system demonstrates strong potential for ML-driven, explainable intrusion detection, several limitations must be acknowledged regarding dataset fidelity, generalisation, and real-world deployability. These factors constrain the conclusions that can be drawn from experimental results and point to areas for future refinement.

6.1 Dataset Limitations

6.1.1 Simulated Environment

CIC-IDS2017 was generated in a controlled lab setting, with scripted user activity and artificial attack events. This introduces a risk of unrealistic traffic distribution and feature patterns that may not reflect actual production environments.

6.1.2 Label Accuracy and Flow Integrity

Studies have identified several flaws in CIC-IDS2017's labelled flow data, including:

- Duplicate and inconsistent flow entries
- Mislabelled or ambiguous flows
- Imperfect protocol recognition

Although reprocessing and cleaning were applied to mitigate these issues, residual artefacts may still affect model behaviour and evaluation reliability.

6.2 Model Limitations

6.2.1 Overfitting to Dataset-Specific Patterns

Despite techniques such as feature selection and stratified sampling, ML models remain vulnerable to overfitting on dataset-specific artefacts — especially if the same dataset is used for both training and testing.

6.2.2 Sensitivity to Feature Noise

Feature values derived from flow exporters may be sensitive to network conditions, timeouts, or collection strategies. Minor differences in collection tools can result in significantly different feature values, which could affect model stability across environments.

6.3 Explainability Constraints

6.3.1 XAI Fidelity and Trust

While SHAP and LIME improve interpretability, their explanations are approximations of model behaviour — not guarantees. There is a risk of misleading analysts if explanation fidelity is low or if outputs are misinterpreted as causal rather than correlative.

6.3.2 Computation Overhead

Generating local explanations for every alert introduces significant computational cost, particularly for SHAP. In real-time deployments, this could introduce latency or necessitate explanation sampling rather than full coverage.

6.4 Operational and Architectural Limitations

- The system operates in a batch inference mode, not in real-time packet capture or live flow ingestion.
- The API and dashboard components are suitable for controlled environments but may require further hardening (e.g., access control, logging policies) for deployment in enterprise networks.
- The absence of live feedback or analyst correction mechanisms limits the system’s adaptability to new threats or concept drift.

6.5 Evaluation Limitations

- No external dataset was used for cross-dataset generalisation testing.
- Latency, throughput, and resilience under high traffic loads remain to be tested at scale.
- Manual validation of XAI explanations was qualitative, not benchmarked against analyst judgments or labelled ground-truth explanations.

6.6 Summary

The system offers a technically sound, interpretable, and modular NIDS foundation, but its scope is constrained by dataset realism, deployment environment, and the offline nature of evaluation. These limitations are acknowledged and will inform the proposed directions for future enhancement.

7. Conclusion and Future Work

This chapter outlines the anticipated outcomes of the proposed system and identifies future directions for development and evaluation. While implementation is ongoing, the architectural design, dataset strategy, and methodology establish a clear foundation for building an interpretable, machine learning-based network intrusion detection system (NIDS).

7.1 Project Summary

This project sets out to design and build a modular flow-based NIDS that integrates supervised machine learning with explainable artificial intelligence (XAI). The system is intended to address two persistent gaps in modern intrusion detection: (1) the lack of flexibility and generalisability in signature-based methods, and (2) the opacity of black-box ML models in security-critical environments.

CIC-IDS2017 was selected as the dataset, with planned reprocessing to improve flow quality and ensure cleaner model training. The proposed detection layer uses Random Forest and XGBoost classifiers, chosen for their balance between performance and explainability. SHAP and LIME are selected as the explanation engines to provide meaningful, human-readable insights into alert decisions.

Monitoring and analyst interaction are planned through a FastAPI backend and a Prometheus-Grafana dashboard. This infrastructure is designed to support real-time flow submission, alert logging, and visual explanation delivery.

7.2 Project State and Next Steps

At this stage, the system architecture, data pipeline design, and evaluation strategy are complete. The next development milestones include:

- Finalising preprocessing scripts and executing feature extraction from CIC-IDS2017 PCAPs
- Training and validating the ML models on cleaned flow data
- Integrating SHAP and LIME into the inference pipeline
- Deploying the FastAPI server and connecting metrics to Grafana for visual testing
- Running performance, inference latency, and explanation response tests

These steps will complete the transition from design to operational prototype.

7.3 Future Development Opportunities

Beyond the scope of the current project timeline, several enhancements can be considered:

- Real-Time Packet Ingestion: Extend the system to capture and classify traffic live using PCAP interfaces or NetFlow exporters.
- Adversarial Hardening: Incorporate techniques such as adversarial training or detection of evasive flows.
- Feedback Loop for Analysts: Build mechanisms for analysts to flag or relabel alerts, enabling continual model refinement.
- Expanded Dataset Testing: Apply the pipeline to newer or cross-domain datasets (e.g. CIC-IoT-DIAD, CSE-CIC-IDS2018) to assess generalisability.
- Lightweight Model Deployment: Explore model compression or edge deployment for low-latency environments.

7.4 Final Remarks

The project, once complete, will deliver a working prototype of a flow-based NIDS that not only detects threats but also explains its decisions in a form analysts can interpret and trust. By prioritising transparency, modularity, and practical deployment features, the system is designed to bridge the gap between academic machine learning and real-world security operations.

References

1. Kamboj, Arvind & Ravindran, Vasudev & Ojha, Sharad. (2025). A Comparative Analysis of Signature-Based and Anomaly-Based Intrusion Detection Systems. International Journal of Latest Technology in Engineering Management & Applied Science. 14. 209-214. 10.51583/IJLTEMAS.2025.140500026.

2. A Comparative Analysis of Signature-Based and Anomaly-Based Intrusion Detection Systems (RePEc:bjb:journl:v:14:y:2025:i:5:p:209-214)by Vasude Karthik Ravindran & Sharad Shyam Ojha & Arvind Kamboj https://ideas.repec.org/a/bjb/journl/v14y2025i5p209-214.html

3. Hozouri, A., Mirzaei, A. & Effatparvar, M. A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges. *Discov Artif Intell* 5, 314 (2025). https://doi.org/10.1007/s44163-025-00578-1

4. Diana, L.; Dini, P.; Paolini, D. Overview on Intrusion Detection Systems for Computers Networking Security. *Computers* 2025, *14*, 87. https://doi.org/10.3390/computers14030087

5. Gutierrez-Garcia, Jose & Sánchez-DelaCruz, Eddy & Pozos-Parra, Maria. (2023). A Review of Intrusion Detection Systems Using Machine Learning: Attacks, Algorithms and Challenges. 10.1007/978-3-031-28073-3_5.

6. Adenusi, D.A., Oladimeji, O.O., Oyekola, T.A. and Olagunju, K.S., 2025. Data-driven network intrusion detection using optimized machine learning algorithms. *Franklin Open*, 12, p.100339. https://doi.org/10.1016/j.fraope.2025.100339

7. Zibi, M.V. and Obagbuwa, I.C., 2025. Evaluating machine learning-based intrusion detection systems with explainable AI: enhancing transparency and interpretability. *Frontiers in Computer Science*, 7. https://doi.org/10.3389/fcomp.2025.1520741

8. Gutierrez-Garcia, Jose & Sánchez-DelaCruz, Eddy & Pozos-Parra, Maria. (2023). A Review of Intrusion Detection Systems Using Machine Learning: Attacks, Algorithms and Challenges. 10.1007/978-3-031-28073-3_5.

9. Hozouri, A., Mirzaei, A. & Effatparvar, M. A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges. Discov Artif Intell 5, 314 (2025). https://doi.org/10.1007/s44163-025-00578-1

10. Khraisat, A., Gondal, I., Vamplew, P. et al. Survey of intrusion detection systems: techniques, datasets and challenges. Cybersecur 2, 20 (2019). https://doi.org/10.1186/s42400-019-0038-7

11. Kumar, A.; Gutierrez, J.A. Impact of Machine Learning on Intrusion Detection Systems for the Protection of Critical Infrastructure. Information 2025, 16, 515. https://doi.org/10.3390/info16070515

12. Canadian Institute for Cybersecurity, 2017. CICIDS 2017 dataset. University of New Brunswick. Available at: https://www.unb.ca/cic/datasets/ids-2017.html

13. Rosay A., Cheval E., Carlier F. and Leroux P. (2022). Network Intrusion Detection: A Comprehensive Analysis of CIC-IDS2017. In Proceedings of the 8th International Conference on Information Systems Security and Privacy - Volume 1: ICISSP, ISBN 978-989-758-553-1, pages 25-36. DOI: 10.5220/0010774000003120

14. Dube, Rohit. (2022). Faulty Use of the CIC-IDS 2017 Dataset in Information Security Research. 10.13140/RG.2.2.25435.64809.

15. Adewole, K.S.; Jacobsson, A.; Davidsson, P. Intrusion Detection Framework for Internet of Things with Rule Induction for Model Explanation. Sensors 2025, 25, 1845. https://doi.org/10.3390/s25061845

16. Samout, N., Gouasmi, T. and Nasri, N., 2025. Explainable AI-based innovative models for intrusion detection in Big Data WSN. Procedia Computer Science, 270, pp.5065–5072. https://doi.org/10.1016/j.procs.2025.09.633

17. Hozouri, A., Mirzaei, A. & Effatparvar, M. A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges. Discov Artif Intell 5, 314 (2025). https://doi.org/10.1007/s44163-025-00578-1

18. Arreche, O., Guntur, T. and Abdallah, M., 2024. XAI-based Feature Selection for Improved Network Intrusion Detection Systems. arXiv [preprint]. Available at: https://arxiv.org/abs/2410.10050

19. Hermosilla, P.; Berríos, S.; Allende-Cid, H. Explainable AI for Forensic Analysis: A Comparative Study of SHAP and LIME in Intrusion Detection Models. Appl. Sci. 2025, 15, 7329. https://doi.org/10.3390/app15137329

20. Hermosilla, Pamela & Berríos, Sebastián & Allende-Cid, Héctor. (2025). Explainable AI for Forensic Analysis: A Comparative Study of SHAP and LIME in Intrusion Detection Models. Applied Sciences. 15. 7329. 10.3390/app15137329.